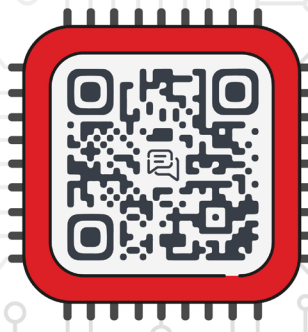




6



10



1500 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Graph



د. مدحت الصوص

RB Informatics; 17/5/2026

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

محتويات المحاضرة:

- تعريف ال Graph وأنواعه.
- البيان البسيط Simple Graph.
- البيان الجزئي Subgraph.
- درجة العقدة $Deg(v)$.
- مفهوم المسار والمسار البسيط.
- مفهوم الحلقة Cycle.
- عدد الوصلات الأعظمي في ال Graph.
- البيان المتصل بقوة Strongly Connected Graph.
- المركبة المتصلة بقوة Strongly Connected Component.
- تمثيل ال Graph حاسوبياً.

مقدمة:

في المحاضرة السابقة، قمنا بدراسة بنية الشجرة (Tree) وكيفية تنظيم العقد والعلاقات بينها، أما اليوم فسنتعرف على بنية أكثر مرونة وانتشاراً في علوم الحاسوب، وهي ال Graph.

Graph:

- يستخدم لتمثيل العلاقات والارتباطات بين العناصر، وبعكس الأشجار (Trees)، لا يوجد في ال Graph مفهوم الأب والابن، ولا يفرض قيوداً محددة على شكل العلاقات كما ان الشجرة تعتبر حالة خاصة من ال Graph.

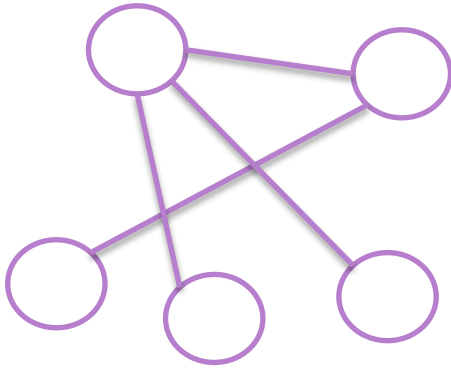
ملاحظات:

- قد يحتوي على وصلات متعددة بين العقد بحسب نوع ال Graph.
- لا يشترط وجود عقدة جذر (Root) كما في الأشجار.

يتكون ال Graph من:

$$G = (V, E)$$

- V تمثل مجموعة العقد.
- E تمثل مجموعة الوصلات التي تربط بين العقد.



شكل ال Graph:

$$V = \{v_1, v_2, \dots\}.$$

$$E = \{(v_1, v_2), (v_2, v_5)\}.$$

أمثلة واقعية على ال Graph:

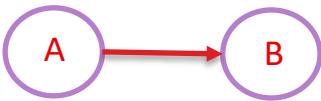
- الساحات والطرق (كل ساحة تعتبر عقدة والشوارع تعبر عن الوصلات بين هذه العقد).
- Instagram, Facebook

{ انا بحدد نوع الوصلات والعقد مثلاً:
 الوصلة: User
 العقدة: Follow }

اهم أنواع ال Graph:

1. بيان موجه (Directed Graph):

في هذا النوع تكون كل الوصلات موجهة، أي لكل وصلة نقطة بداية ونقطة نهاية. ويُعبّر عن الاتجاه باستخدام سهم يحدد مسار الانتقال بين العقد. ولكي يُسمى ال Graph موجهاً، يجب أن تكون جميع الوصلات فيه موجهة.



2. بيان غير الموجه (Undirected Graph):

في هذا النوع تكون جميع الوصلات غير موجهة، أي لا يوجد اتجاه محدد للانتقال بين العقد. فإذا كانت العقدة A متصلة بالعقدة B، فإن الانتقال ممكن في كلا الاتجاهين. ولكي يُعتبر ال Graph غير موجهاً، يجب أن تكون جميع الوصلات فيه غير موجهة.



لم نبدأ بعد ^_^

3. بيان موزون (Weighted Graph):

في هذا النوع تحتوي الوصلات على أوزان، أي أنني سأقوم بإضافة بيانات (Data) أو قيم معينة على الوصلات لتعبر عن معلومات معينة مثل:

1. طول الطريق
2. الزمن
3. التكلفة
4. المسافة

مثال:

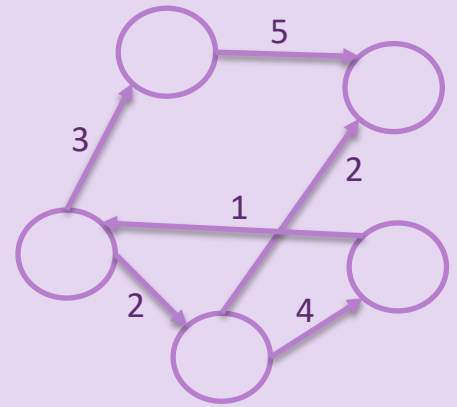
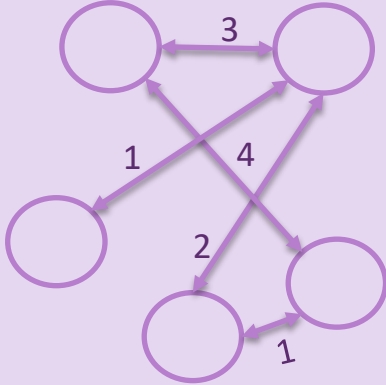
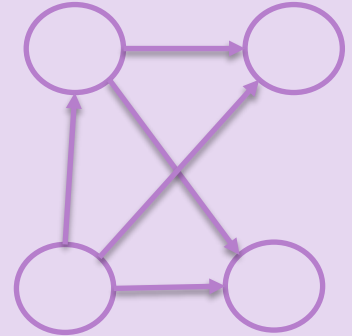
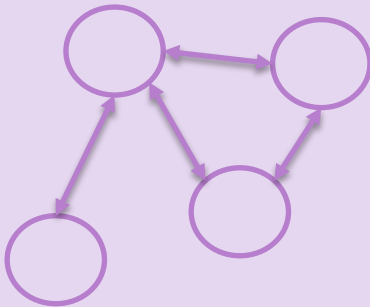
إذا كانت الوصلة بين مدينتين تحمل الرقم 10، فقد يمثل ذلك ان المسافة بينهما تساوي 10 كم،  ولكي يُسمى ال Graph موزوناً، يجب أن تكون جميع الوصلات فيه مرتبطة بأوزان أو بيانات.

4. بيان غير موزون (Unweighted Graph):

في هذا النوع لا تحتوي الوصلات على أي بيانات إضافية أو أوزان، وإنما تمثل مجرد علاقة أو اتصال بين العقد. ولكي يُسمى ال Graph غير موزوناً، يجب أن تكون جميع الوصلات فيه غير مرتبطة بأوزان أو بيانات.



■ امثله توضح الأنواع الأربعة لل Graph التي قمنا بدراستها:

1. بيان موجه موزون.2. بيان غير موجه موزون.3. بيان موجه غير موزون.4. بيان غير موجه غير موزون.

ملاحظة: الأشجار هي بيان موجه غير موزون لأنها تحتوي على وصلات بدون اوزان وباتجاه واحد.

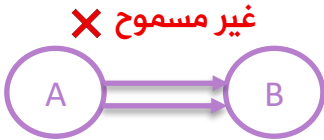
مفاهيم وتعريفات أساسية:

قبل البدء بدراسة الخوارزميات والتطبيقات المتعلقة بال Graph، سنتعرف أولاً على مجموعة من المفاهيم والتعريفات المهمة لفهم طريقة عمل ال Graph.

1. البيان البسيط (Simple Graph):

يسمى ال Graph بياناً بسيطاً، عندما يحقق الشرطين التاليين:

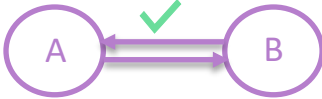
1. لا يمكن وجود أكثر من وصلة وبنفس الاتجاه بين نفس العقدتين.



2. عدم وجود وصلة ذاتية (أي لا يمكن ان ترتبط العقدة بنفسها).



■ ملاحظة: إذا كان يوجد وصلتان بين نفس العقدتين، ولكن بعكس الجهة فإن ال Graph يبقى بسيطاً.



2. البيان الجزئي (Subgraph):

البيان الجزئي هو Graph يتم الحصول عليه من Graph اساسي أكبر، بحيث نأخذ جزءاً من العقد والوصلات الموجودة في ال Graph الاصلي. أي ان جميع العقد والوصلات الموجودة Subgraph يجب ان تكون مسبقاً موجودة في ال Graph الاساسي.

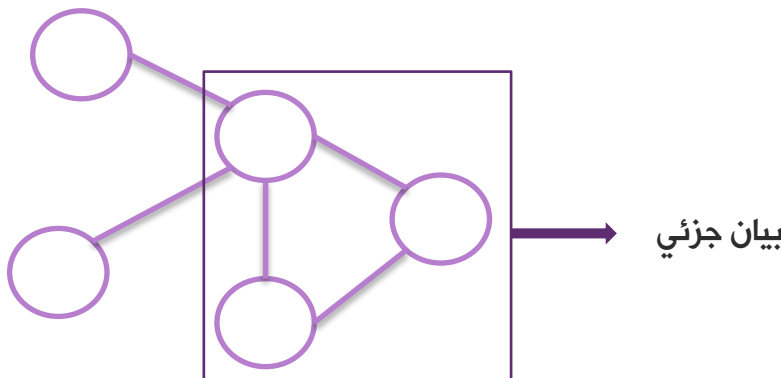
■ ليكن لدينا البيان $G = \langle V, E \rangle$ نقول إن البيان $G' = \langle V', E' \rangle$ هو بيان جزئي له إذا تحقق:

$$V' \subseteq V, E' \subseteq E - 1$$

جميع العقد والوصلات تنتمي للبيان الأساسي.

$$\forall (a, b) \in E' \longrightarrow a, b \in V' - 2$$

حيث a, b عقد و (a, b) هي وصلة بين العقدتين، أي ان كل وصلة في هذا البيان يجب أن ينتمي طرفاها إلى العقد الجزئية.



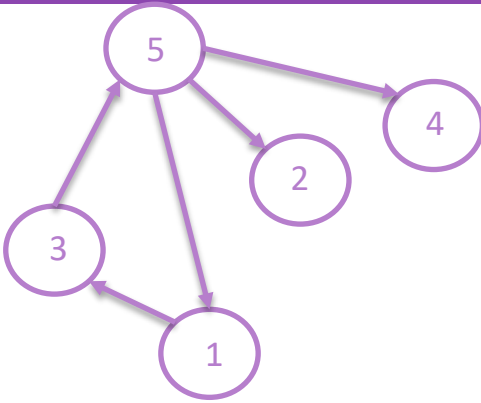
درجة العقدة $Deg(v)$: هو عدد الوصلات المجاورة (الصادرة والواردة) لهذه العقدة.

▪ $Deg^-(v)$: عدد الوصلات الواردة إلى هذه العقدة.

▪ $Deg^+(v)$: عدد الوصلات الخارجة من هذه العقدة.

ودرجة العقدة بشكل عام هي مجموع الوصلات الواردة والخارجة.

$$Deg(v) = Deg^-(v) + Deg^+(v)$$



مثال:

▪ $Deg^+(5) = 3$

▪ $Deg^-(5) = 1$

▪ $Deg(5) = 3 + 1 = 4$

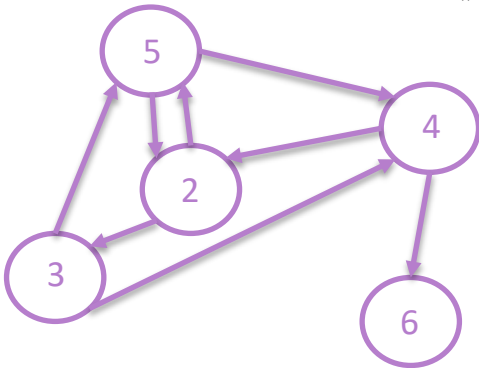
ملاحظة: في البيان الموجه: $Deg(v) = Deg^-(v) + Deg^+(v)$

في البيان غير الموجه: $Deg(v) = Deg^-(v) = Deg^+(v)$

↪ المسار (Path):

المسار في ال Graph هو تسلسل من العقد المتصلة ببعضها عن طريق الوصلات، بحيث يمكن الانتقال مع عقدة إلى أخرى عبر هذه الوصلات. أي ان كل عقدتين متتاليتين في المسار يجب أن يكون بينهما وصلة مباشرة

مثال:



واحد من المسارات هو:



مسار آخر:

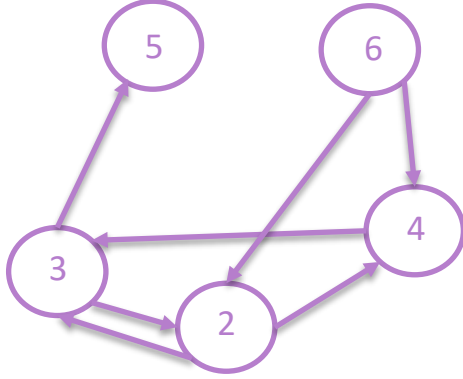


إذ لم تنجح من أول محاولة حاول مرة أخرى...

المسار البسيط (Simple Path):

يسمى المسار مساراً بسيطاً، إذا لم تتكرر فيه أي عقدة أثناء المرور ضمن المسار أي أن كل عقدة تظهر مرة واحدة فقط وبالتالي لا يحتوي المسار على حلقات (Cycles أو Loop).

مثال:

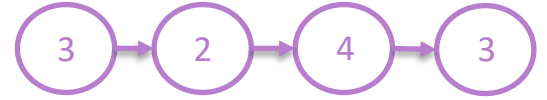
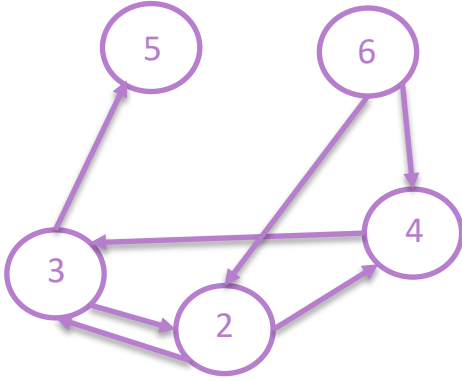


لم تتكرر أي عقدة

الحلقة (Cycle):

هي مسار تبدأ من عقدة معينة وتنتهي عند نفس العقدة، مع المرور بعدة عقد أو وصلات بينهما.

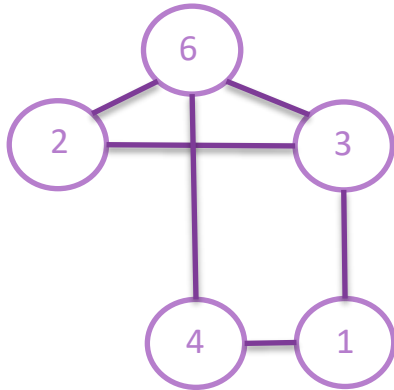
مثال:



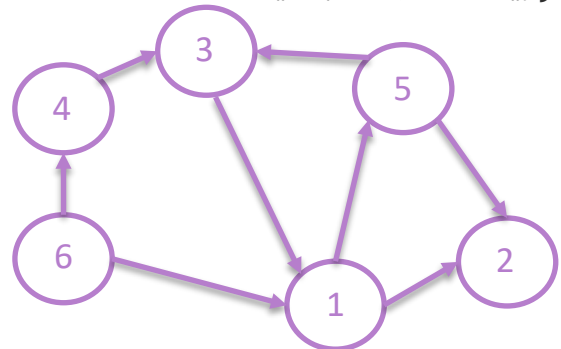
بدأنا من (3) وانتهينا عنده لذلك سُمي هذا المسار حلقة.

البيان المتصل بقوة (Strongly Connected Graph):

- يمكن الانتقال فيه من أي عقدة إلى أي عقدة أخرى، والعكس صحيح (أي عقدتين في البيان، يجب أن يوجد مسار بينهما في الاتجاهين).



هذا بيان متصل بقوة

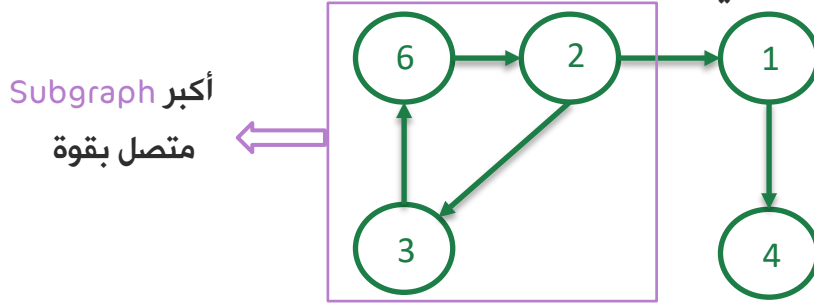


ليست بيان متصل بقوة

لا يوجد مسار يمكنني الانتقال فيه إلى العقدة 6

المركبة المتصلة بقوة (Strongly Connected Component):

هو أكبر Subgraph متصل بقوة في ال Graph الأساسي.



ملاحظة: العقدة (1) لا يمكن اعتبارها ضمن أكبر Subgraph متصل بقوة، لأن لا يمكنها الوصول إلى جميع العقد (لا يمكنها الوصول إلى العقدة التي قبلها).
لأن ال Graph الأساسي هو Directed Graph.

تمثيل ال Graph حاسوبياً

عندما نريد تمثيل ال Graph داخل الحاسوب وتخزينها، لا نملك سوى أدوات بسيطة مثل المصفوفات (Arrays) والمؤشرات (Pointers)، لذلك نحتاج طريقة منظمة لتمثيل العقد والوصلات.

1. الطريقة الأولى:

مصفوفة التجاور:

- هي مصفوفة ثنائية الأبعاد حجمها $(n \times n)$ ، حيث n هو عدد العقد في ال Graph.
- وكل عنصر في المصفوفة يمثل وجود أو عدم وجود وصله بين عقدتين.
- إذا أردنا تخزين اتصال بين عقدتين نستخدم اتجاه المصفوفة من الأسطر إلى الأعمدة، بحيث:
- السطر يمثل العقدة المصدر.
- العمود يمثل العقدة الهدف.

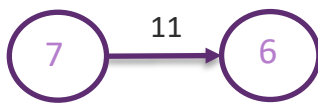
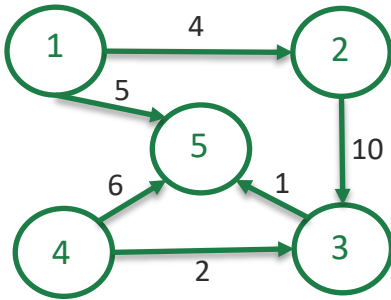
مثال: إذا كان لدينا وصلة من العقدة 1 إلى العقدة 2 وكانت هذه الوصلة موجودة، فإننا نضع القيمة True في الخانة $arr[1][2] = \text{Tree}$ ، أما إذا لا توجد وصلة فنضع False.

ملاحظة: إذا كانت ال Graph موزونه نضع قيمه الوصلة في مكانها (بدلاً من True).



Mistakes are proof that you are trying...

■ لنفرض لدينا هذا ال Graph :



■ عدد العقد هو (7) لذلك نشكل مصفوفة ثنائية مكونة من (7×7) .

■ نبدأ من السطر الأول:

- العقدة (1) مرتبطة مع العقدة (2)

بوصلة وزنها (4)، لذلك عند السطر (1)

في عمود العقدة المشار إليها في

نهاية الوصلة (وهي العقدة 2) نضع الوزن.

- العقدة (2) مرتبطة مع العقدة (3)

بوصلة وزنها (1)، لذلك عند سطر العقدة (2)

نضع وزن الوصلة في العمود الذي يحوي

العقدة (3) في حالتنا العمود الثالث.

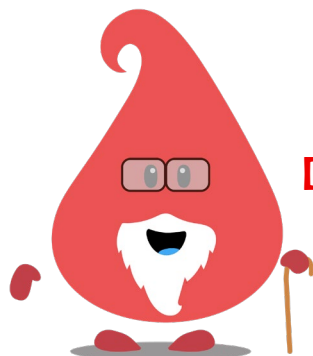
- وهكذا نكمل مع باقي العقد.

	1	2	3	4	5	6	7
1		4			5		
2			1 0				
3					1		
4			2		6		
5							
6							
7						1 1	

ملاحظة:

■ نضع وزن الوصلة في مكان التقاطع ما بين عقدة البداية وعقدة النهاية في البيان الموزون.

■ أما في البيان الغير موزون نضع القيم (True, False).



Don't watch the clock, do what it does.
Keep going...

من أبرز مشاكل استخدام مصفوفة التجاور إنها تستهلك ذاكرة كبيرة، لأننا نخصص مصفوفة بحجم $(n \times n)$ حتى لو كان عدد الوصلات الفعلية قليل جداً وهذا يعني يوجد فراغات كثيرة داخل المصفوفة (أي هدر في الذاكرة).

والحل:

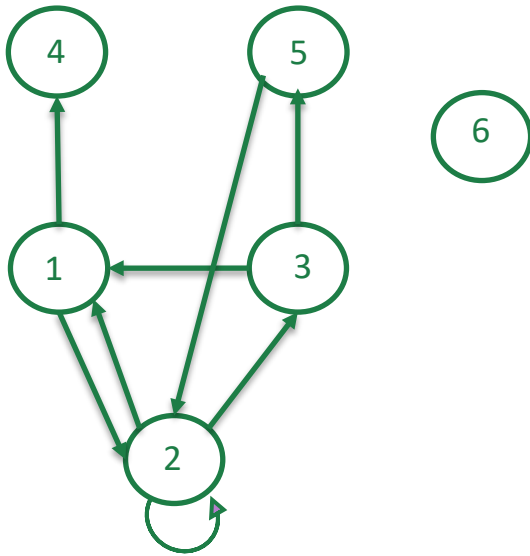
2. الطريقة الثانية:

سلاسل التجاور:

- بدل أن نخزن مصفوفة كاملة لكل العقد، نقوم بتخزين: لكل عقدة، نخزن فقط العقد المرتبطة بها (المجاورات).
- نقوم بتعريف مصفوفة احادية عناصرها في LinkedList ويكون عدد عناصر هذه المصفوفة بنفس عدد عقد ال Graph.
- في كل عنصر من المصفوفة (أي في كل LinkedList)، نخزن الوصلات والعقد المجاورة للعقدة الأساسية.

مثال:

■ لنفرض لدينا هذا ال Graph:



■ عدد العقد هو (6) لذلك نحجز مصفوفة احادية عناصرها هي LinkedList وعدد عناصرها هو 6.

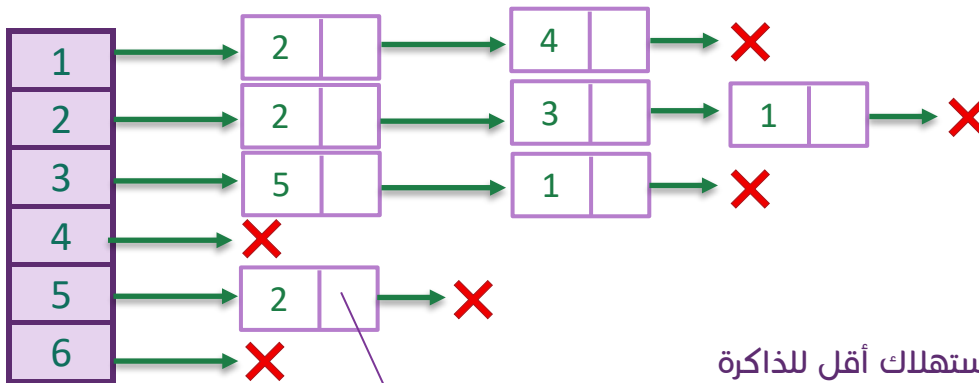
■ العقدة (2):

مجاوراتها: العقدة (2) و (3) و (1).
وبالتالي ال LinkedList تمثل العقد المجاورة للعقدة الأساسية (2).

■ العقدة (1):

مجاوراتها: العقدة (4) و (2).
وبالتالي ال LinkedList تمثل العقد المجاورة للعقدة الأساسية (1).

■ وهكذا نكمل باقي العقد.

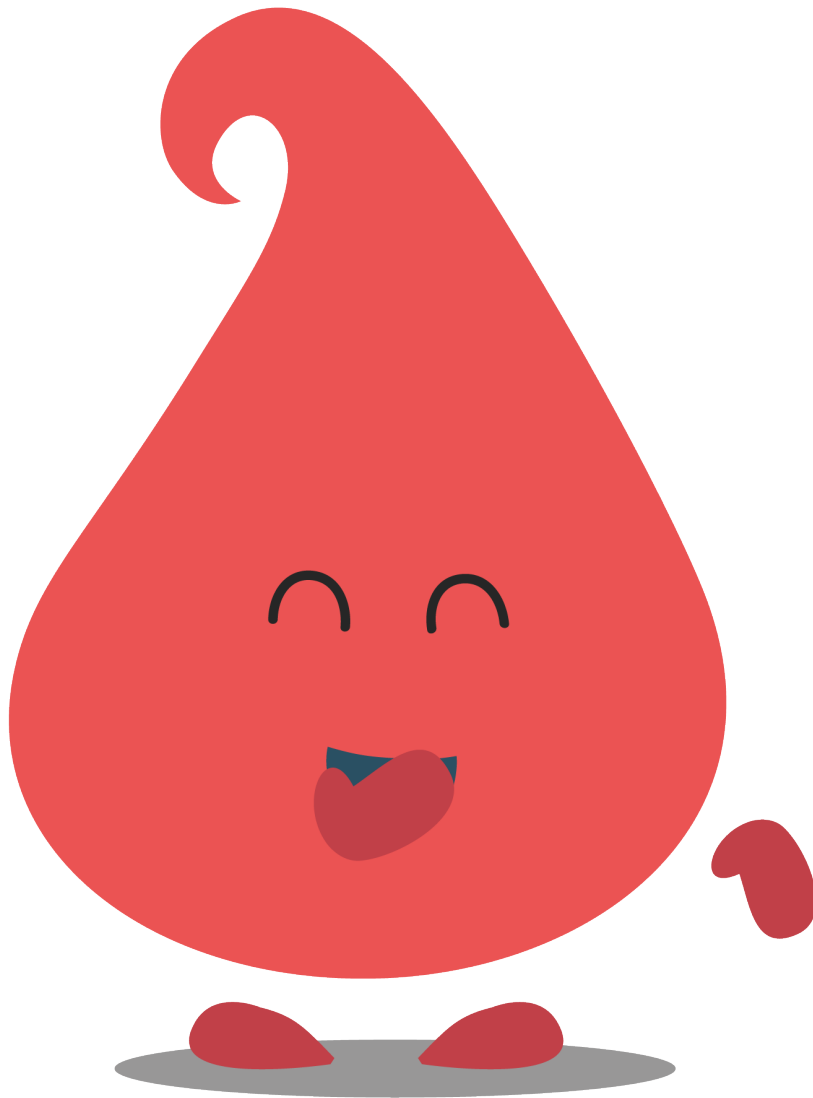


نلاحظ استهلاك أقل للذاكرة

يمكن تخزين قيمة الوصلة

ملاحظة:

يمكن استبدال مصفوفة التجاور ب HashMap لتخزين علاقات ال Graph بشكل أكثر مرونة، خاصة عندما تكون العقد غير رقمية.



إذا كانت الحياة عبارة عن شجرة، فتأكد من أنك تستخدم خوارزمية

ال Graph لتجنب التعثر في الفروع.